



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Department of Computer Science
Faculty of Computing

SECP3843 – SPECIAL TOPIC IN DATA ENGINEERING

(Tutorial 2)

Creating a Simple ETL Pipeline with Apache Spark

Programme	: Bachelor of Computer Science (<i>Data Engineering</i>)
Subject Code	: SECP3843
Subject Name	: Special Topic in Data Engineering
Session-Sem	: 2025/2026-02

Prepared by : 1) Muhammad Naim bin Abdullah (A23CS0134)
2) Muhammad Adam bin Razali (A23CS0116)
3) Afif Shaqir Irfan bin Arqam (A23CS0204)

Section : 02

Lecturer : Dr Aryati binti Bakri

1.0 Business Understanding	3
1.1 Domain Context	3
1.2 Business Objectives	3
1.3 Success Criteria	3
2.0 Data Understanding	4
2.1 Data Source Description	4
2.2 Data Quality Observation	5
2.3 Relational Data Model	6
3.0 Data Preparation	8
3.1 Ingestion	8
3.2 Transformation	9
4.0 Modelling	11
4.1 System Architecture	11
4.2 Multidimensional Data Model (Star Schema)	14
4.3 Justifications of Design Decisions	18
5.0 Evaluation	18
5.1 Pipeline Execution Metrics	18
5.2 Metabase Dashboard Visualization	19
5.3 Limitation	20
5.4 Improvement	20
6.0 Reflection	21

1.0 Business Understanding

1.1 Domain Context

The Brazilian Basic Education Census, known as Censo Escolar, is a massive yearly survey run by the government to track the country's school system. It gathers information from every public and private school in Brazil, covering everything from student details and teacher count to school resources like internet, libraries, and water. This data is released for free to the public, providing an incredibly detailed look at education across the country.

1.2 Business Objectives

1. Speed Up Research :

Instead of spending hours manually opening files, researchers should be able to get answers in seconds using a fast database

2. Ready to use dashboards :

Make the data easy to view in visualisation tools like Power BI without needing extra technical setup

1.3 Success Criteria

The pipeline will be judged successful if all of the following are true :

- All 12 dimension tables and the 1 main fact table exist in the target database and properly linked together.
- All 12 years data from 2010 until 2021 are loaded, with per-year row counts matching the source CSVs
- Make sure every data that are present is accurate for example in year 2021 will have the highest student enrollment

2.0 Data Understanding

2.1 Data Source Description

The dataset used in this pipeline is the Microdados do Censo Escolar da Educação Básica 2022, published by the Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira (INEP), an agency of the Brazilian Ministry of Education. It is released annually as part of the national Basic Education Census (Censo Escolar), which is the most comprehensive school-level survey conducted in Brazil.

The dataset was downloaded from the Brazilian Government's Open Data Portal (<https://www.gov.br/inep/pt-br/aceso-a-informacao/dados-abertos/microdados>) as a compressed ZIP archive, then extracted and normalised into a single flat CSV file named `microdados_ed_basica_2021.csv`. The file uses a semicolon (;) as its column delimiter and is encoded in ISO-8859-1 (Latin-1) to accommodate special Portuguese characters.

Key characteristics of the raw source file are summarised in the table below:

Attribute	Details
File Name	<code>microdados_ed_basica_2021.csv</code>
Year	2021
Total Records (Schools)	224,649 rows
Total Columns	385 attributes
File Encoding	ISO-8859-1 (Latin-1)
Delimiter	Semicolon (;)
Data Publisher	INEP / Brazilian Ministry of Education
Granularity	One row per school

The 385 columns cover various thematic groups: school identification, location, administration, physical infrastructure, and facilities (libraries, labs, courts). They also track technology/internet access, disability accessibility, educational modalities, class schedules, and key metrics like total

enrollments (QT_MAT_BAS), teachers (QT_DOC_BAS), classes (QT_TUR_BAS), and student computers (QT_COMP_ALUNO).

The code snippet below, taken from `etl_pipeline.py`, shows how Spark ingests the raw file:

```
file_path = r"C:\censo-project\microdados_censo_escolar_2022\Microdados do Censo Escolar da Educação Básica 2022\dados\microdados_ed_basica_2022.csv"

df_raw = spark.read.csv(file_path, header=True, sep=";", encoding="iso-8859-1")
```

2.2 Data Quality Observation

The INEP Censo Escolar data isn't just raw or crowd-sourced info, it is officially checked and verified by the government. Before releasing the yearly ZIP files, INEP runs its own checks to fix formatting, make sure numbers make sense, and ensure all files match up perfectly. Because of this, we can treat the downloaded data as a clean, trusted source. Our ETL pipeline doesn't need to clean up messy data and its main job is just to restructure that big flat CSV file into a neat star schema.

During the data exploration phase, we noticed a few practical things about how the data is set up:

- Yes/No Flags (Prefixed with IN_): Columns like IN_BIBLIOTECA (library) or IN_INTERNET use 1 for yes and 0 for no. If a school didn't answer, the space is just left blank.
- Count Numbers (Prefixed with QT_): For columns like QT_MAT_BAS (total students) or QT_DOC_BAS (total teachers), spaces are left empty instead of showing a 0 or NULL if a school didn't report any numbers.
- Categories (Prefixed with TP_): These use number codes instead of text. For example, for TP_DEPENDENCIA (school administration type), 1 means Federal, 2 means State, 3 means Municipal, and 4 means Private.
- Dates: The start and end dates of the school year (DT_ANO_LETIVO_INICIO and DT_ANO_LETIVO_TERMINO) use an old-school SAS format (like 14APR2022:00:00:00). Since this isn't the usual standard format (ISO-8601), you might need to convert it later when writing queries.

- **Missing Info:** The IN_AGUA_FILTRADA (filtered water) column was completely blank in our sample rows, which suggests this specific field might be optional or wasn't collected for the 2022 school year.

In a production-grade architecture, a dedicated data quality layer (the Bronze-to-Silver transition in the Medallion architecture) would sit before this pipeline to handle null imputation, outlier detection, and type casting.

2.3 Relational Data Model

The source data is delivered as a single denormalised flat file where every school's attributes geographic location, infrastructure, facilities, equipment, educational modality, and numeric metrics appear as columns in the same row. While convenient for export, this structure is inefficient for analytical queries because it requires scanning hundreds of columns and many rows repeatedly for every aggregation.

To address this, the pipeline transforms the flat file into a Star Schema into a relational modelling pattern optimised for analytical workloads. A star schema organises data into one central Fact Table surrounded by several smaller Dimension Tables. The fact table records measurable, numeric events (the 'what happened' and 'how many'), while dimension tables store descriptive context (the 'who, where, and what kind').

The relational model implemented in this pipeline consists of twelve dimension tables and one fact table:

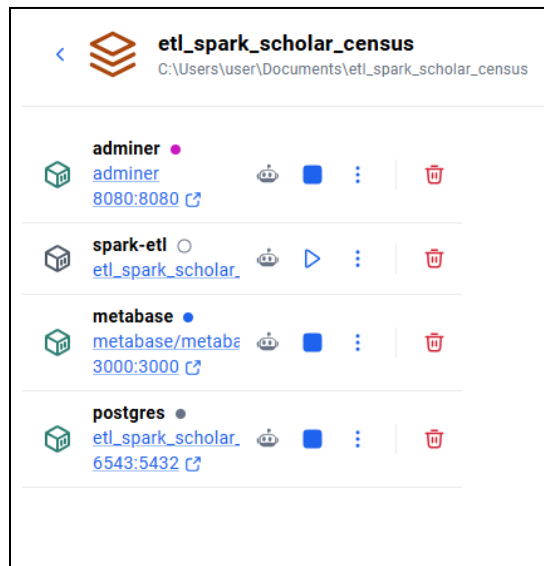
Dimension Table	Key Columns	Description
dim_location	CO_REGIAO, CO_UF, CO_MUNICIPIO	Geographic region, state, and municipality
dim_school	CO_ENTIDADE, NO_ENTIDADE, TP_DEPENDENCIA, TP_LOCALIZACAO	School identity, admin type, and urban/rural
dim_infrastructure	IN_AGUA_FILTRADA, IN_ENERGIA_REDE_PUBL	Water, electricity, sewage, waste disposal

	ICA, IN_ESGOTO_REDE_PUBLI CA, IN_LIXO_ENTERRA	
dim_facilities	IN_BIBLIOTECA, IN_LABORATORIO_CIENC IAS, IN_LABORATORIO_INFOR MATICA, IN_QUADRA_ESPORTES_ COBERTA	Library, labs, sports court
dim_equipment	IN_COMPUTADOR, IN_INTERNET, IN_BANDA_LARGA, IN_EQUIP_LOUSA_DIGIT AL	Computers, internet, broadband, smart board
dim_accessibility	IN_DEPENDENCIAS_PNE, IN_BANHEIRO_PNE	Facilities for students with disabilities
dim_health_nutrition	IN_ALIMENTACAO, IN_AGUA_POTAVEL	Canteen/meals and potable water
dim_education_stage	IN_REGULAR, IN_ESP, IN_EJA, IN_PROF	Regular, special, adult, and vocational ed.
dim_class_schedule	TP_PROPOSTA_PEDAGOG ICA, TP_AEE	In-person, blended, remote, and night-shift
dim_teacher_profile	TP_PROPOSTA_PEDAGOG ICA, TP_AEE	Pedagogical model and special ed. support
dim_indigenous_program	IN_EDUCACAO_INDIGEN A, TP_INDIGENA_LINGUA, IN_LOCAL_FUNC_GALPA O	Indigenous education and language type
dim_time	NU_ANO_CENSO, DT_ANO_LETIVO_INICIO, DT_ANO_LETIVO_TERMI NO	Census year and academic calendar dates

3.0 Data Preparation

All of the preparation steps are implemented through the `run_etl.py` where it combined both the ingestion and transformation levels of processing. It is also invoked inside a Docker Container alongside other dependencies such as Spark runtime, Postgres JDBC, and Python to allow the output to be shared among them.

For all of the Ingestion and Transformation process, all of the necessary tools are containerized inside docker which include postgres, metabase, spark-etl, and adminer as seen below:



3.1 Ingestion

```
def main() -> None:
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    for year in YEARS:
        download_year(year)
    print("[done] downloads complete", flush=True)
```

Before Spark reads the data, the raw data from INEP releases must be downloaded, unzipped, and normalized into a predictable filename based on the year of the records. By default, the data

is pulled from the year 2010 until 2021 and stored inside data/*.csv path. This process simplifies the querying of the data for ingestion.

3.2 Transformation

CSV to Parquet Conversion

```
data_csv = (  
    spark.read.format("csv")  
    .option("header", "true")  
    .option("inferSchema", "true")  
    .option("delimiter", ";")  
    .option("encoding", "latin1")  
    .load("/app/data/*.csv")  
)
```

Then the stored data let's Spark pickup through it by the number of years, then encode it with latin 1 encoding and semicolon(;) delimiter to read the data row by row based on the column available.

```
parquet_path = "/app/data/censo_escolar.parquet"  
log(f"writing parquet -> {parquet_path}")  
data_csv.write.mode("overwrite").parquet(parquet_path)
```

From the earlier .csv file that has been read by Spark, it is turned into a Parquet which is a columnar binary format. It is converted to allow for smaller size in comparison with csv, and better querying performance.

Preparing and Creating for Fact and Dimension table Conversion

```
INTEGER_DIMENSIONS = [
    "TP_DEPENDENCIA",
    "TP_LOCALIZACAO",
    "IN_AGUA_POTAVEL",
    "IN_ENERGIA_INEXISTENTE",
    "IN_ESGOTO_INEXISTENTE",
    "IN_BANHEIRO",
    "IN_BIBLIOTECA",
    "IN_REFEITORIO",
    "IN_COMPUTADOR",
    "IN_INTERNET",
    "IN_EQUIP_NENHUM",
]

DIMENSION_TABLES_CONFIG = {
    "DIM_LOCAL": {
        "fields": [
            {"field": "NO_UF", "type": "string"},
            {"field": "SG_UF", "type": "string"},
            {"field": "CO_UF", "type": "string"},
            {"field": "NO_MUNICIPIO", "type": "string"},
            {"field": "CO_MUNICIPIO", "type": "string"},
        ]
    }
}

DIMENSION_TABLES_CONFIG.update(
    {
        "DIM_" + dim.upper(): [{"field": dim, "type": "integer"}]
        for dim in INTEGER_DIMENSIONS
    }
)
```

Dimension table is declared based on the integer dimensions generated, becoming the single-column dimension via the name DIM <COLUMN>. After that we run the code to convert the parquet created into fact and table dimensions via this code below into the Postgres database:

```
for table_name, table_config in DIMENSION_TABLES_CONFIG.items():

    print(f"[{datetime.now()}] Writing {table_name}")

    data\
    .select(
        [
            F
            .col(field["field"])
            .cast(field["type"])
            .alias(field["field"])

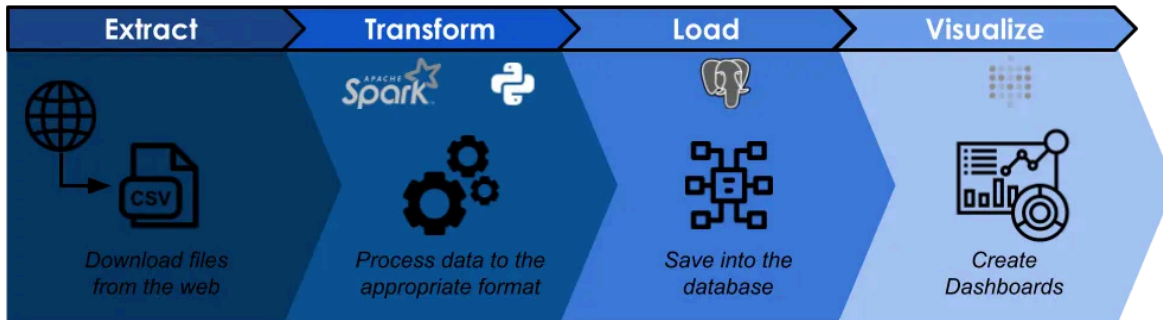
            for field
            in table_config["fields"]
        ]
    )\
    .distinct()\
    .withColumn(
        "id", F.monotonically_increasing_id()
    )\
    .write\
    .jdbc(
        **POSTGRES_CONFIG,
        table=table_name,
        mode="overwrite"
    )

    print(f"[{datetime.now()}] Wrote {table_name}")
    # Define id as the primary key
    cursor = conn.cursor()
    cursor.execute(
        f"ALTER TABLE {table_name} ADD PRIMARY KEY (id);"
    )
    cursor.close()
    conn.commit()

    print(f"[{datetime.now()}] Added primary key to {table_name}")
    print(f"[{datetime.now()}] Done")
```

4.0 Modelling

4.1 System Architecture



The system is built as a local, single-node data pipeline that combines Apache Spark for large-scale data transformation, PostgreSQL as the target relational data warehouse, and Python as the orchestration language via the PySpark library. The architecture follows a classic Extract–Transform–Load (ETL) pattern divided into four sequential phases, as illustrated below.

Phase 1

Extract: The raw CSV file is read directly from the local disk using Spark's CSV reader. The encoding (ISO-8859-1) and delimiter (semicolon) options are passed explicitly to handle the format of the INEP release. Spark does not load the data into memory at this point; it merely builds a logical execution plan, a key advantage over Pandas for large files.

```
df_raw = spark.read.csv(file_path, header=True, sep=";", encoding="iso-8859-1")
```

Phase 2

Transform (Dimensions): For each of the twelve dimension themes, a distinct subset of columns is extracted from `df_raw` using `.select().distinct()`, and a surrogate integer key is appended with `monotonically_increasing_id()`. This creates twelve dimension DataFrames in memory (as lazy plans).

```
# 1. Location
dim_location = df_raw.select("CO_REGIAO", "CO_UF", "CO_MUNICIPIO").distinct() \
    .withColumn("id_location", monotonically_increasing_id())
```

Phase 3

Transform (Fact Table): The fact table is assembled by chaining twelve left joins one per dimension back on the original raw DataFrame. Each join matches the natural key columns of that dimension and brings in the surrogate key. The final `.select()` retains only the twelve surrogate foreign keys and the four numeric measures.

Phase 4

Load: All thirteen DataFrames (twelve dimensions plus one fact) are written to a PostgreSQL instance via the JDBC connector using `df.write.jdbc()` in overwrite mode. Spark automatically drops and recreates each table upon writing, eliminating the need for pre-existing DDL scripts.

```
for table_name, df in tables_to_write.items():
    print(f"    -> Saving {table_name}...")
    df.write.jdbc(url=DB_URL, table=table_name, mode="overwrite", properties=DB_PROPERTIES)
```

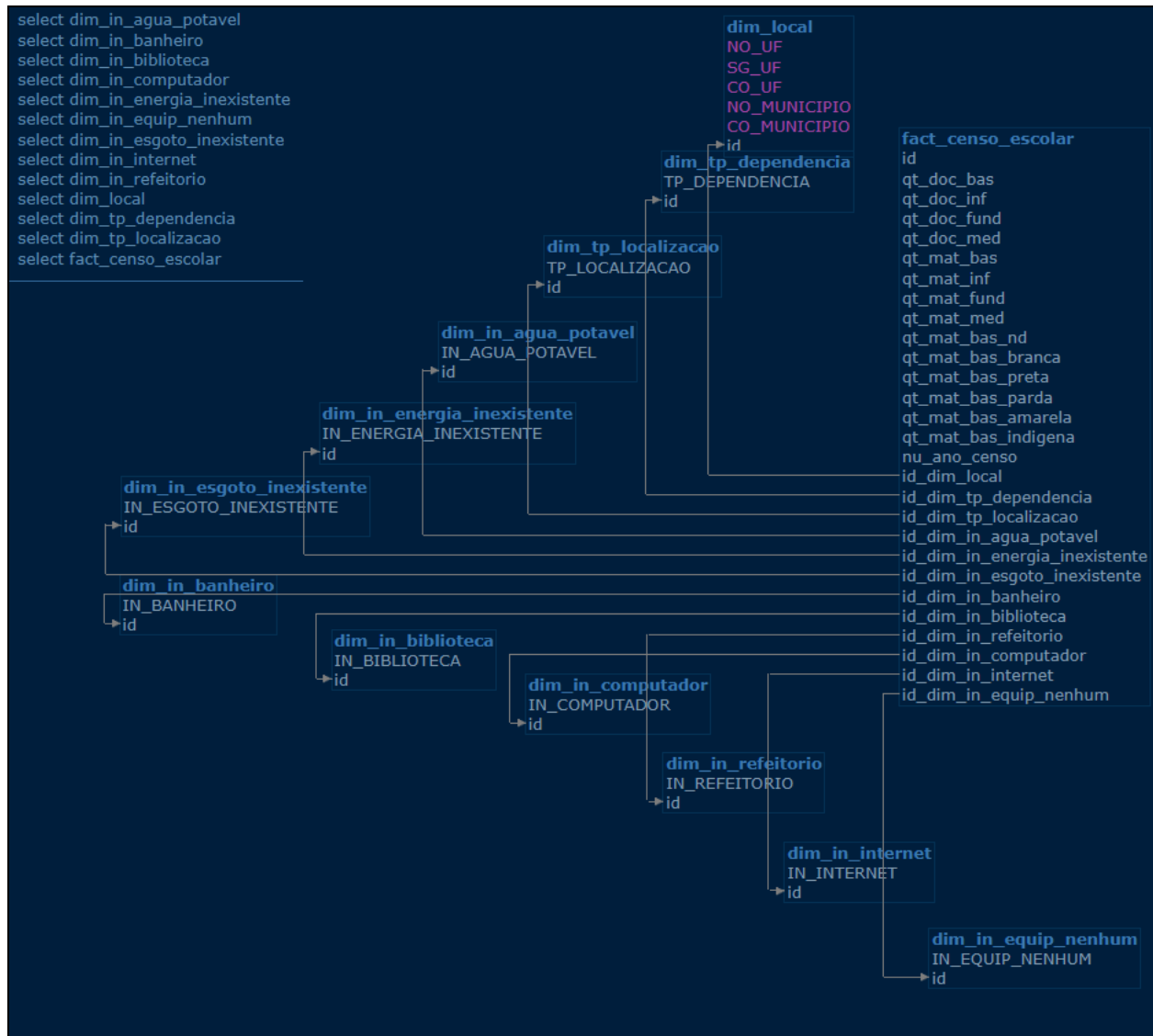
The overall technology used in the pipeline is as table below:

Component	Technology / Details
Data Processing	Apache Spark 3.x via PySpark
Orchestration Language	Python
Data Source	INEP CSV (ISO-8859-1, semicolon-delimited)
Database	PostgreSQL (censo_escolar database)
DB Connectivity	PostgreSQL JDBC Driver (postgresql-42.6.0.jar)
Environment	Docker (Spark + Postgres + Python dependencies)
Visualization	Metabase (connects to PostgreSQL)

4.2 Multidimensional Data Model (Star Schema)

From the tutorial, we were able to derive a star schema that denormalizes the geographic hierarchy into a single DIM_LOCAL and turns each indicator flag into a one-column dimension. Each analytical query became a fact table with N number of dimension joins on integer keys which is equal to 12. The implementation of the star schema is solely to improve the analytical query of the data since the relational model is only appropriate for the transactional updates, but perform poorly for analytical purposes.

Star schema



Fact Table Information

fact_censo_escolar	Rows	id	SERIAL PK	Surrogate row id
	2,792,984	nu_ano_censo	INT	Census year (2010–2021) — degenerate dimension
		qt_doc_bas	INT	Teacher count, basic ed total
		qt_doc_inf	INT	Teacher count, early-childhood
		qt_doc_fund	INT	Teacher count, primary
		qt_doc_med	INT	Teacher count, secondary
		qt_mat_bas	INT	Enrollment; basic ed total (use this for headline figures)
		qt_mat_inf	INT	Enrollment, early-childhood
		qt_mat_fund	INT	Enrollment, primary
		qt_mat_med	INT	Enrollment, secondary
		qt_mat_bas_nd	INT	Enrollment; race not declared
		qt_mat_bas_branca	INT	Enrollment, white
		qt_mat_bas_preta	INT	Enrollment, black
		qt_mat_bas_parda	INT	Enrollment, mixed/brown
		qt_mat_bas_asiatica	INT	Enrollment; East-Asian descent
		qt_mat_bas_indigena	INT	Enrollment, indigenous
		id_dim_local	BIGINT FK	→ dim_local(id)
		id_dim_tp_dependencia	BIGINT FK	→ dim_tp_dependencia(id)
		id_dim_tp_localizacao	BIGINT FK	→ dim_tp_localizacao(id)
		id_dim_in_agua	BIGINT	→ dim_in_agua_potavel(id)

		_potavel	FK	
		id_dim_in_energia_inexistente	BIGINT FK	→ dim_in_energia_inexistente(id)
		id_dim_in_esgoto_inexistente	BIGINT FK	→ dim_in_esgoto_inexistente(id)
		id_dim_in_banheiro	BIGINT FK	→ dim_in_banheiro(id)
		id_dim_in_biblioteca	BIGINT FK	→ dim_in_biblioteca(id)
		id_dim_in_refeitorio	BIGINT FK	→ dim_in_refeitorio(id)
		id_dim_in_computador	BIGINT FK	→ dim_in_computador(id)
		id_dim_in_internet	BIGINT FK	→ dim_in_internet(id)
		id_dim_in_equip_nenhum	BIGINT FK	→ dim_in_equip_nenhum(id)

Dimension Table Information

Dimension tables	Rows	Columns	Meaning
dim_local	5500	id, "NO_UF", "SG_UF", "CO_UF", "NO_MUNICIPIO", "CO_MUNICIPIO"	Geography: state name / abbrev / IBGE code + municipality name / IBGE code. Use for geo breakdowns and choropleths.

dim_tp_dependencia	4	id, "TP_DEPENDENCIA"	Administrative type: 1=Federal; 2=Estadual (state); 3=Municipal; 4=Privada
dim_tp_localizacao	2	id, "TP_LOCALIZACAO"	1=Urbana; 2=Rural
dim_in_agua_potavel	2	id, "IN_AGUA_POTAVEL"	Has potable water (0/1)
dim_in_energia_inexistente	2	id, "IN_ENERGIA_INEXISTENTE"	Lacks electricity (0/1) — reverse polarity; watch out
dim_in_esgoto_inexistente	2	id, "IN_ESGOTO_INEXISTENTE"	Lacks sewage (0/1) — reverse polarity
dim_in_banheiro	2	id, "IN_BANHEIRO"	Has bathroom (0/1)
dim_in_biblioteca	2	id, "IN_BIBLIOTECA"	Has library (0/1)
dim_in_refeitório	2	id, "IN_REFEITORIO"	Has cafeteria (0/1)
dim_in_computador	2	id, "IN_COMPUTADOR"	Has computer (0/1)
dim_in_internet	2	id, "IN_INTERNET"	Has internet (0/1)
dim_in Equipamento nenhum	2	id, "IN_EQUIP_NENHUM"	Has no equipment at all (0/1)

4.3 Justifications of Design Decisions

- **One fact per school and not one fact per enrollment.** Since the source granularity is at the school level, drilling up the into city, as well as state level can be computed cheaply, therefore optimizing resource allocation.
- **Dimension table is separated into 12 instead of one wide infrastructure.** Since the source already provided a flag which contributed to one generation of dim per flag automatically.
- **Year is utilized as a degenerate dimension.** Meaning that no DIM_YEAR needed to be created to access the data by year since the cardinality is one row per year. This is standard Kimball practice.

5.0 Evaluation

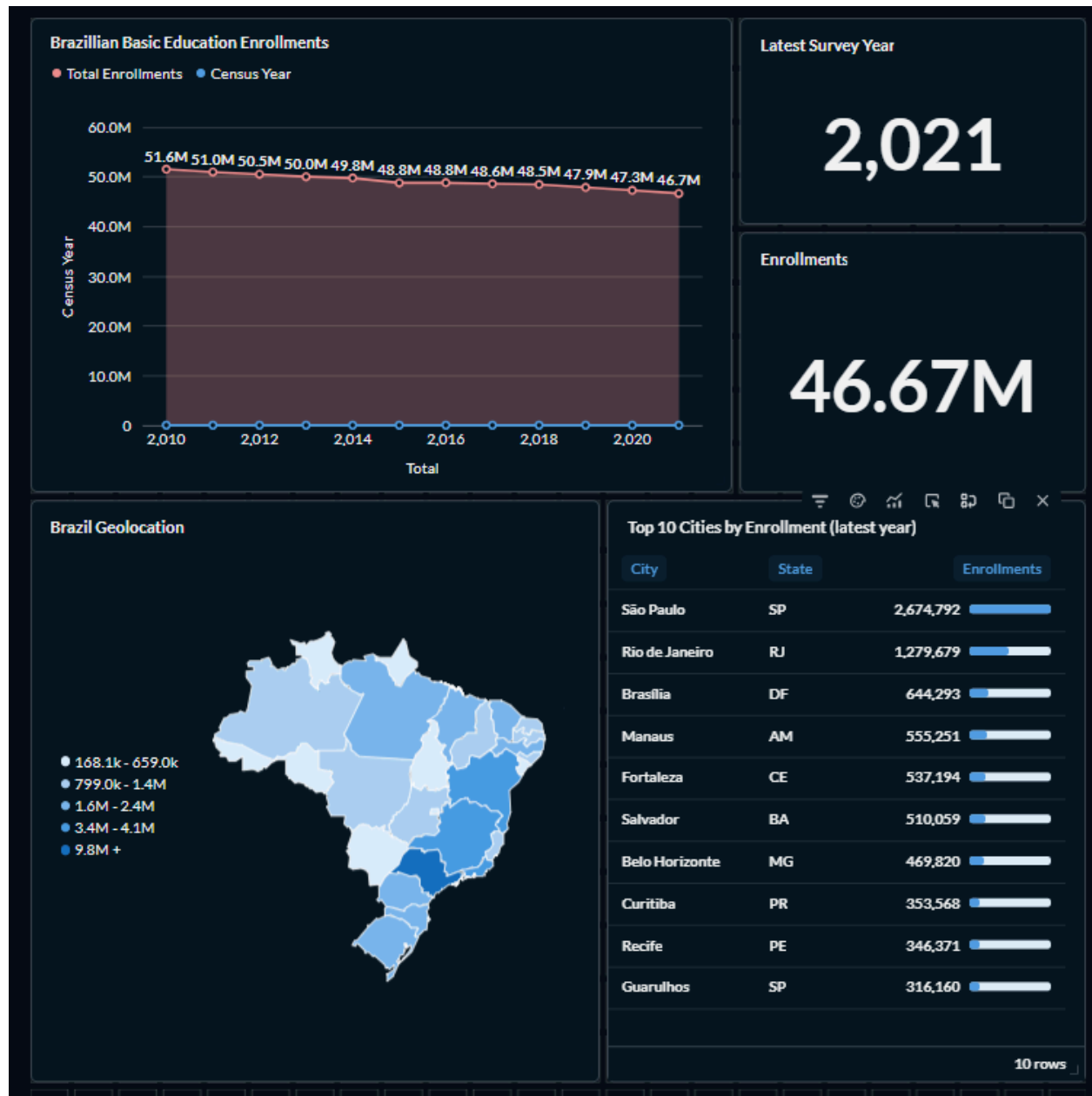
This section discusses the results of the pipeline runtime, limitations and improvements in the pipeline.

5.1 Pipeline Execution Metrics

Here is the estimation of the pipeline execution time from ingestion until end of transformation:

Stage	Total Runtime
CSV download (12 zips, ~2.2 GB) — cold	~25 – 30 min (network bound)
CSV download — warm (all years cached)	< 1 s (idempotent skip)
12-CSV → Parquet	~90 s
12 dimension writes (incl. ADD PRIMARY KEY)	~10 s total
Fact-table DDL	< 1 s
Fact-table load (2.79 M rows, 12 joins)	~4 min
Total end-to-end — cold	~30 min
Total end-to-end — warm (CSVs already on disk)	~5 – 6 min

5.2 Metabase Dashboard Visualization



5.3 Limitation

While The pipeline works well for this project, there are a few technical issues to keep in mind :

- Starts Over Every Time :

The system deletes and recreates the entire database every time it runs. There is no features to add the data for a new year, you need to reload all previous years from scratch

- Trusts The Source Data :

The pipeline assumed that the raw files were already perfect. For good practice, we would suggest adding “checkpoints” to catch the errors such as missing values, etc before they reach the master database.

- Limited Capacity :

The data processing (Apache Spark) happens on a single computer. This may work for 2.79 million rows, but if the dataset grew to more than 50 million rows, we would need to use a multiple computer (a cluster) to handle the load

5.4 Improvement

This are the suggestion to make this project from just a tutorial to a production ready system

- Smart Updates :

Change the system so it can add a new year at a time without needing to reprocess the entire history. This will help to save a lot of time.

- Automated Quality Checks :

Use specialized tools to automatically detect any weird data or errors the moment they appear.

- Cloud Power :

Move the processing to a cloud service like Databrick. This will allow the system to handle data from every school in the world and no need to care about the storage.

6.0 Reflection

Muhammad Naim bin Abdullah

Through this project, I gained hands-on experience building an end-to-end ETL pipeline using Apache Spark and Python. I developed a workflow that automatically extracts a CSV data (2010–2021) from a web source. I then designed and executed a data transformation strategy, structuring the raw data into a star schema consisting of 1 fact table and 12 dimension tables. Finally, I loaded the processed data into a PostgreSQL database. Overall, this project significantly deepened my practical knowledge of data engineering principles and ETL architecture.

Muhammad Adam bin Razali

For this project, I got to do most of the technical stuffs including setups for Spark, Postgres, Metabase and Docker. What excites me is on the pipeline creation where we ingest and transform the data via a python code where I link my local code with the docker container via a dockerfile and docker compose.yml. It was a nightmare of a setup by I enjoy the end product of it and now understand how docker works as a container. I am also able to touchdown on the dashboard creation inside the metabase where I linked all the processed data stored inside the Postgres into the metabase via a driver.

Afif Shaqir Irfan bin Arqam

This was my first time using Apache Spark to go through a full ETL process in data engineering. I started by understanding the data source before processing it with Apache Spark, which handles the ETL transformations in a distributed manner. From there, I learned how Python holds everything together as the pipeline script and controls the overall flow. I also got hands-on experience setting up a PostgreSQL JDBC connector to load the transformed data into a PostgreSQL database called censo_escolar, structured as a star schema. The entire environment was containerised using Docker Compose, which kept Spark, PostgreSQL, and Python running together in one place without any compatibility headaches. Finally, Metabase was connected to the database to build dashboards and visualise the data in a meaningful way. Overall, it was a

great end-to-end experience that showed me how all these tools fit together in a real data engineering pipeline.